

AMOS: A Memory Operating Layer for Autonomous Data Analysis

Abstract

This document is the product architecture and build guide for amos. It defines the theory, system boundaries, data model, runtime protocol, component interfaces, security model, deployment topology, operating model, and phased implementation plan needed to build amos from an empty repository to a production service. The intended reader is a startup engineering team that must make the system work, operate it safely, and know when each layer is complete.

AMOS is a model-neutral operating layer between autonomous data-analysis applications and a company's data systems. It converts changing schemas, metrics, snapshots, documents, permissions, prior analyses, reviewer corrections, and execution records into governed analytical memory. For each task, it compiles a bounded working set, grants scoped tool capabilities, validates the versions used, verifies the computation, records claim-level support, and commits the artifact with a replay and invalidation contract. Models propose plans and interpretations; deterministic services retain authority over access, execution, validation, and publication.

The architecture is deliberately incremental. A startup first ships a modular monolith that completes one read-only analytical workflow end to end. It then adds durable feedback, scheduling, multi-tenant governance, connector breadth, and controlled autonomy only after explicit correctness, security, reliability, and customer-value gates pass. The result is a complete engineering blueprint rather than a research agenda: every major responsibility has an owner, interface, persistent record, failure behavior, test strategy, and build criterion.

1 Purpose, Audience, and Definition of Complete

1.1 Product Definition

AMOS is the control and memory plane for autonomous data analysis. It is not a warehouse, a dashboard, a general document search engine, or a particular language model. It is the layer that makes analytical work persistent, permissioned, temporally valid, inspectable, and repeatable across models and applications.

The product has three surfaces over one substrate:

1. **Analytical memory:** a typed and versioned representation of the company's metrics, schemas, data states, documents, policies, decisions, prior work, and feedback.
2. **Agent runtime:** a governed execution environment for planning, SQL and Python tools, verification, artifact construction, review, scheduling, and recovery.
3. **Application platform:** APIs and templates for recurring business reviews, investigations, monitors, forecasts, planning workflows, and domain-specific analytical agents.

1.2 Who Uses This Guide

The primary readers are the founding engineer, data-systems engineer, agent-runtime engineer, product engineer, security owner, and design-partner lead. Sections 2–5 define why the system works. Sections 6–12 define what to build. Sections 13–16 define how to ship, operate, test, and expand it.

1.3 Completeness Standard

The architecture is complete only when an engineer can answer all of the following without inventing a new subsystem:

- where every governing definition, permission, execution, claim, and artifact is stored;
- how identity and policy are enforced before model-visible context is constructed;
- how a task moves from admission to publication, abort, repair, or human review;
- how source versions are observed and revalidated at commit time;
- how every important claim resolves to computation and governing state;
- how feedback changes future work without silently rewriting history;

- how a source change invalidates dependent conclusions;
- how the service is deployed, monitored, recovered, and tested;
- which capabilities are intentionally deferred and what gate unlocks them.

2 The Product Problem and System Boundary

An analytical task depends on independently changing state. At time t :

$$\mathcal{S}_t = (q_t, D_t, C_t, M_t, X_t, H_{<t}, F_{<t}, P_t), \quad (1)$$

where q_t is the request or trigger, D_t is the data snapshot or stream position, C_t is schema and catalog state, M_t is metric and semantic state, X_t is governed document state, $H_{<t}$ is prior analytical work, $F_{<t}$ is reviewed feedback, and P_t is identity and policy.

These sources do not share one transaction manager. A metric can change after retrieval, a table can migrate during execution, a watermark can advance, and a permission can be revoked before publication. A correct-looking answer can therefore be state-invalid even when its SQL ran successfully. The product problem is to control this cross-source state transition without copying all company data into a prompt or replacing the systems of record.

2.1 Responsibilities

AMOS owns:

- typed metadata, governed summaries, external references, aliases, versions, authority, and supersession;
- identity-aware context construction and task-scoped capabilities;
- plan, tool, verification, claim, artifact, review, replay, and invalidation records;
- a stable runtime contract above replaceable models, indexes, warehouses, and application surfaces.

Source systems continue to own raw data, primary access control, warehouse transactions, catalog records, semantic definitions, document originals, and identity truth. Applications own user experience and domain workflow. Models own no durable authority: model output is a proposal until a deterministic gate accepts it.

2.2 First Product Scope

The first production slice supports one tenant, one warehouse, one approved metric family, read-only SQL, governed documents, charts, report claims, reviewer approval, and replay. It excludes arbitrary production writes, unrestricted notebook code, general multi-agent scheduling, unreviewed causal claims, and autonomous external communication. Expansion occurs only through the gates in Section 18.

3 Theory: Analytical Memory as an Operating Layer

3.1 Memory Object

Persistent analytical memory is a set $\mathcal{M}_t$ of typed objects. Each object is:

$$m = (id, k, v, \tau_e, \tau_r, a, p, s, r, h), \quad (2)$$

where k is type, v is content or an external reference, τ_e is the effective-time interval, τ_r is recording time, a is authority, p is permission scope, s is lifecycle status, r is provenance, and h is a content hash. Logical identity and physical version are separate: multiple versions can describe one metric or schema, but only versions valid for the task interval and policy may govern a run.

Core types are data or stream state, schema/catalog state, semantic definition, governed document, prior analysis, feedback, permission policy, execution, artifact, claim, and provenance edge. Raw fact tables remain external; memory stores identifiers, governed summaries, versions, and immutable references.

3.2 Bounded Active Context

The model-visible working set $\mathcal{W}_t \subset \mathcal{M}_t$ is compiled for one task. It is not ordinary top- k retrieval. Selection must satisfy:

$$\text{permitted}(u, p_t, m) \quad \forall m \in \mathcal{W}_t, \quad (3)$$

$$\text{effective}(m, t_q) \wedge \text{active}(m), \quad \forall m \in \mathcal{W}_t, \quad (4)$$

$$\sum_{m \in \mathcal{W}_t} \text{tokens}(m) \leq B, \quad (5)$$

$$\forall r \in R(q) : \exists m \in \mathcal{W}_t \text{ covering role } r. \quad (6)$$

The required-role set $R(q)$ typically includes metric, schema, data state, policy, and supporting evidence. A missing mandatory role produces an explicit non-final outcome rather than a guessed answer. This is the central scale separation: persistent memory may grow to millions of objects while model context remains bounded by B .

3.3 Authority, Time, and Supersession

Recency alone does not determine truth. The default authority order is owner-approved > reviewer-approved > user note > model hypothesis > untrusted external content. Effective time determines when a statement applies; recording time determines when amos learned it. Supersession edges explicitly replace one logical version with another. Conflicting owner-approved objects for the same interval block publication until resolved.

3.4 Consistency Classes

Each connector declares the strongest attainable observation class:

- **C0 observed:** the version and observation time are recorded, but the source cannot guarantee re-readability.
- **C1 validated:** the version can be re-read or compared at commit, so changes are detectable.
- **C2 retained:** the exact version is retained or leased long enough for deterministic reconstruction.

Task policy declares minimum classes per required role. Regulated or high-impact artifacts may require C2 data and C1 policy/semantic state; exploratory tasks may accept C0 with a visible warning.

4 The Analytical State Transaction

A-TXN is the system contract connecting memory, tools, validation, publication, and later invalidation. It is not an ACID transaction across external systems. It creates a truthful AMOS commit record after optimistic revalidation of the external versions used.

4.1 Transaction Record

An atxn record contains:

$$A = (id, u, q, P^0, V^0, \mathcal{W}, K, E, C, \mathcal{G}, o, t), \quad (7)$$

where P^0 is the admitted policy epoch, V^0 is the source-version vector, K is the scoped capability set, E is execution evidence, C is the claim set, $\mathcal{G}$ is the dependency graph, o is outcome, and t contains lifecycle timestamps.

4.2 Lifecycle

1. **BEGIN:** authenticate identity, bind tenant, classify risk, assign budgets, record policy epoch, and create an idempotent transaction ID.
2. **OBSERVE:** ask connectors for source versions, freshness, retention class, and access capabilities.
3. **SELECT:** filter candidates before ranking; reconcile identity, time, authority, and supersession; require role completeness.

4. **PLAN:** produce a typed plan with declared inputs, tool operations, expected outputs, budgets, and review obligations.
5. **EXECUTE:** issue short-lived capabilities to isolated workers; verify each operation before execution; record parameters, hashes, timings, and results.
6. **COMPOSE:** generate artifact structure and extract policy-relevant claims.
7. **VERIFY:** validate SQL, schema, metrics, freshness, permissions, claim support, and risk-specific rules.
8. **REVALIDATE:** re-read the policy epoch and every C1 version; confirm C2 retention leases.
9. **COMMIT:** atomically store artifact metadata, claims, dependencies, verification, replay descriptor, and audit event.
10. **INVALIDATE:** on later source events, traverse reverse dependencies and mark claims stale, revoked, or review-required.

4.3 Runtime Invariants

The implementation must enforce these invariants:

1. no inaccessible memory enters model-visible context;
2. every selected object is effective, active, non-superseded, and source-qualified;
3. every tool call is allowed by both policy and a task-scoped capability;
4. execution references only declared working-set objects and observed source versions;
5. numeric claims have computation, data-state, schema, and metric support;
6. causal or high-impact recommendations remain review-gated unless an approved design permits automation;
7. commit fails or becomes explicitly non-final when governing C1 state changed;
8. memory writes preserve authority, history, source, and the transaction that produced them;
9. every final artifact has a declared replay level and complete audit record.

4.4 Outcome State Machine

Outcome	Meaning	Required behavior
pass	All required checks succeed.	Commit as final within the declared risk class and replay level.
warning	Work is structurally valid with a disclosed limitation.	Commit as non-final or limited-use; surface the exact warning and affected claims.
repair	A bounded, policy-safe correction is possible.	Record failed attempt, patch the plan or operation, and retry within budget.
needs_review	Evidence exists but judgment or approval is required.	Commit evidence and draft artifact; create a review task; prohibit final publication.
reject	Policy, safety, validity, or support fails.	Store the rejection and diagnostics; do not publish the artifact.
abort	Infrastructure, lease, source, or concurrency failure prevents a truthful decision.	Preserve partial evidence, release capabilities, and allow idempotent retry.

Table 1: Every task ends in an explicit state; silence and best-effort finalization are forbidden.

5 End-to-End Product Architecture

The startup should implement the first release as a modular monolith with strict internal interfaces. This provides one transactional database, one deployment unit, and one debugging surface while preserving seams for later service extraction.

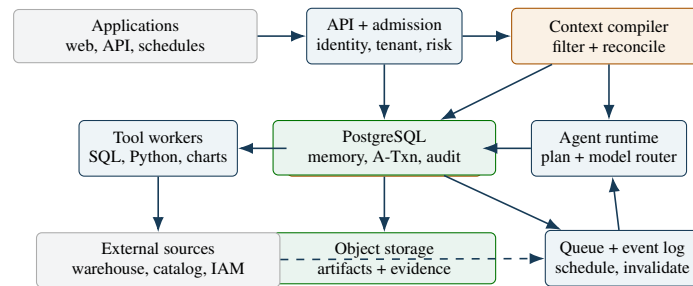


Figure 1: Production logical architecture. Internal boundaries remain explicit even when deployed as one application.

Component	Owns	Must not own
API and admission	Authentication, tenant binding, request validation, idempotency, risk class, quotas.	Prompt construction, warehouse credentials, or final publication logic.
Connector manager	Source discovery, version observation, freshness, access probes, change events, read handles.	Cross-source reconciliation or company-wide policy decisions.
Memory service	Typed objects, versions, authority, effective time, aliases, supersession, indexes.	Arbitrary raw customer data or model prompts as the system of record.
Context compiler	Required-role derivation, policy-first filtering, ranking, reconciliation, compaction, manifest.	Tool execution or hidden fallback when a required role is missing.
Agent runtime	Typed planning, model routing, budgets, checkpoints, repair loop, artifact draft.	Credentials, unrestricted tools, or authority to bypass verifier outcomes.
Tool workers	Sandboxed SQL, Python, statistics, charting, deterministic result capture.	Long-lived tenant secrets or direct publication.
Verifier	SQL safety, schema, metrics, freshness, permissions, claim support, risk and review rules.	General truth or causal certainty beyond declared rules.
Evidence service	Claims, dependency edges, hashes, verification, artifacts, replay descriptors.	Silent mutation of committed evidence.
Scheduler and invalidator	Durable jobs, retries, source-change fan-out, revalidation and review tasks.	Unbounded automatic reruns or publication without current policy.
Review application	Evidence inspection, approve/reject/correct, scoped feedback, audit export.	Direct mutation of historical objects or unscoped high-authority feedback.

Table 2: Ownership boundaries prevent the language model or any single service from becoming an implicit superuser.

5.1 Component Responsibilities

6 Product Surfaces and Operating Workflows

The control plane is useful only when its state is legible to customers. AMOS therefore ships four product surfaces over the same APIs and permission checks; none receives privileged database access.

6.1 Customer Journey

Onboarding is an auditable promotion workflow. An administrator creates the tenant, connects a read-only source, confirms identity synchronization, and chooses retention and region. The connector discovers schemas and capabilities into a staging namespace. A data owner resolves aliases, defines or imports a metric, assigns authority, and approves a version. A product owner then configures an application template with task schema, required roles, risk class, review rule, schedule, budget, and artifact shape. Production activation is prohibited until connector, permission, metric, task-fixture, and recovery checks pass.

During normal use, a request or schedule creates a transaction and immediately displays its policy scope. The workspace streams durable state changes rather than model tokens as the source of truth. A user can open the exact context manifest, each execution, each verifier result, and the dependency chain of every material claim. Final status distinguishes *published*, *published with warning*, *awaiting review*, *rejected*, *stale*, and *revoked*; a draft is never visually equivalent to a governed artifact.

After review, corrections append new governed objects and show their scope before confirmation. The next applicable transaction selects them by authority and effective time. Source or policy change computes impact, marks dependent claims, and either revalidates, schedules bounded re-analysis, or requests review. The user can always answer: what changed, what AMOS did, what remains valid, and who has authority to resolve the rest.

Surface	Primary user	Required capabilities
Memory Studio	Data owner and administrator	Connect and test sources; inspect discovered objects and versions; define aliases, metrics, authority, retention, and policies; approve changes; view source health and impact before activation.
Analysis Workspace	Analyst and business operator	Submit or schedule a typed task; see admitted scope, plan, budget, progress, warnings, and terminal state; inspect an artifact down to claim, computation, and source; export approved results.
Review Queue	Reviewer and domain owner	Compare evidence with the draft; approve, reject, or correct at claim, metric, template, or policy scope; assign effective time and authority; see which future tasks and existing claims are affected.
Operations Console	Support, security, and platform operator	Search transactions and audit events; inspect connector, queue, worker, and policy health; retry an idempotent step, revoke a capability, quarantine a connector, revalidate claims, and run recovery procedures.

Table 3: The product exposes governed state and explicit controls instead of presenting an opaque chat transcript.

6.2 Experience Invariants

Every UI action uses the same idempotent API and produces an audit event. Hidden background work has a visible job and owner. Destructive actions are modeled as revocation, supersession, or policy-governed erasure. Restricted object names and failure details are redacted at the API, not merely in the interface. No screen offers a bypass around review, verification, retention, or tenant boundaries.

7 Data Architecture and Persistent Contracts

7.1 Core Relational Model

Use PostgreSQL as the authoritative store. Object storage holds large artifacts, source exports, chart images, and retained execution payloads. A queue transports durable jobs; it does not replace the transaction log.

Table	Primary identity	Required content and rule
tenants	tenant_id	Status, retention policy, encryption key reference, feature gates. Every governed row includes tenant_id.
identities	tenant_id + subject_id	Provider, roles, groups, policy attributes, last synchronized epoch.
source_systems	source_id	Connector type, configuration reference, attainable consistency classes, health. No plaintext secret.
memory_objects	object_id	Type, logical key, source, authority, permissions, effective interval, recorded time, status, content hash, external reference.
memory_versions	logical_key + version	Immutable content or reference, source version, transaction time, supersedes link. Updates append; they do not overwrite.
alias_edges	edge_id	Source-qualified logical identity mapping, authority, effective interval, reviewer state.
policies	policy_id + epoch	Declarative admission, retrieval, tool, publication, retention, and review rules.
a_transactions	transaction_id	Request, identity, risk, budgets, state, policy epoch, version vector, timestamps, terminal outcome.
context_manifests	manifest_id	Selected object versions, required-role coverage, omissions, conflicts, token counts, ranking trace.

Table	Primary identity	Required content and rule
plans	plan_id	Typed steps, declared inputs/outputs, tools, budgets, dependencies, model/configuration identity.
executions	execution_id	Capability, worker, tool version, parameters hash, input versions, output reference/hash, latency, cost, status.
artifacts	artifact_id	Type, immutable object reference, audience, risk, publication state, creator transaction, current validity.
claims	claim_id	Artifact span, text, claim type, risk, review obligation, verification and validity state.
dependency_edges	edge_id	Claim-to-query, data, metric, schema, document, execution, feedback, or decision relation.
verification_records	verification_id	Check type/version, inputs, result, warnings/errors, repairability, timestamp.
replay_packages	package_id	Replay level, retained inputs, tool/model versions, parameters, hashes, environment descriptor.
reviews	review_id	Reviewer identity, decision, comment, correction scope, authority, affected claims, timestamp.
audit_events	event_id	Append-only actor, action, target, request/transaction IDs, outcome, policy epoch, timestamp.
jobs	job_id	Schedule/trigger, idempotency key, state, retry policy, lease, next run, dead-letter reason.

Table 4: Minimum production schema. JSON payloads may supplement but never replace indexed identity, time, policy, and state columns.

7.2 Memory Object Contract

Every memory object must contain a stable ID, source-qualified logical key, type, human-readable summary, structured payload or external reference, source, authority, effective start/end, recorded timestamp, permission labels, sensitivity, version, status, supersedes/superseded-by links, provenance reference, and content hash. Optional confidence cannot override authority.

Write rules are strict. Connectors may write source-observed objects within their namespace. Reviewed corrections may create reviewer-approved feedback. Only designated owners can create owner-approved definitions. Model-inferred memory starts as a hypothesis and cannot govern a high-risk task until approved. Deletion becomes revocation or tombstoning unless a retention policy requires physical erasure.

7.3 Indexes and Access Paths

The first release uses relational indexes and PostgreSQL full-text search. Required indexes include tenant/type/status, logical key/version, effective interval, permission label, source version, authority, supersession, transaction state, job lease, and reverse dependency target. Vector or hybrid retrieval is an optional candidate generator; permissions, time, authority, and supersession are always re-applied as structural filters.

7.4 Retention and Encryption

Encrypt transport and storage. Use per-environment keys initially and tenant-specific keys for enterprise isolation. Store secret references in a secret manager. Define retention independently for prompts, model responses, query results, raw evidence, artifacts, audit events, and C2 snapshots. A replay claim is allowed only when the required data remains retained for its declared interval.

8 Connector Architecture

Each connector implements a small, testable contract:

- `discover(scope)` returns source-qualified entities and capabilities;
- `observe(ref)` returns version, effective time, freshness, sensitivity, access, and consistency class;
- `read(ref, version, capability)` returns governed content or a bounded read handle;
- `validate(ref, observed_version)` confirms whether a C1 observation still holds;
- `subscribe(cursor)` emits version, schema, permission, freshness, or deletion events;
- `health()` reports authentication, rate limits, lag, and degraded capabilities.

Connector output is untrusted until normalized and policy-labeled. Source text never carries executable policy. Each connector receives least-privilege, tenant-scoped credentials and runs behind egress restrictions. Pagination, retries, backoff, rate-limit state, and source cursors are durable.

8.1 Connector Build Order

Build one warehouse adapter first, then identity/policy, semantic definitions, catalog/schema, documents, and event/stream state. Each new adapter must ship with fixtures, contract tests, access-revocation tests, version-change tests, pagination tests, and a failure-mode table. Do not add a connector solely for breadth; add it when a paid workflow requires one of its roles.

9 Context Compiler

The context compiler turns a task schema into a permission-safe working set and an auditable manifest.

9.1 Compilation Algorithm

1. derive required and optional roles from the task type and risk policy;
2. query candidate indexes with tenant and permission pushdown;
3. remove inaccessible, revoked, expired, and superseded objects before semantic ranking;
4. group by source-qualified logical key and resolve aliases explicitly;
5. reconcile effective time and authority; surface equal-authority conflicts;
6. choose at least one valid object for every mandatory role;
7. fill remaining budget by relevance, authority, freshness, verification state, and diversity;
8. compact only by producing a derived object with links to every source object;
9. emit a context manifest containing selections, omissions, conflicts, warnings, token cost, and source versions.

9.2 Failure Behavior

Missing role coverage yields `codeneeds_review` or `codereject`. Ambiguous approved metrics yield a clarification request. Conflicting owner definitions block. Insufficient token budget never silently removes policy, schema, metric, or data-state roles; it drops optional evidence first. Restricted-object counts and identifiers are not revealed to unauthorized users.

10 Agent Runtime and Tool Execution

10.1 Typed Planning

A plan is data, not free-form prose. Each step declares purpose, required object IDs, tool, parameters schema, expected output, dependency steps, timeout, row/data limits, cost budget, retry policy, verification rules, and review obligation. The runtime validates the plan before any capability is issued.

Model routing is policy-driven. A small classifier may assign task type, a coding model may propose SQL, and a reasoning model may plan an investigation. The stored plan records the exact model provider, model ID, configuration, prompt template version, input manifest, latency, token use, and response hash. Models are replaceable processors; persistent memory and safety controls are not provider-specific.

10.2 Capabilities and Workers

The runtime exchanges a plan step for a short-lived, single-purpose capability. A capability binds tenant, transaction, identity, tool, allowed source, read/write mode, resource limits, expiry, and policy epoch. Workers reject expired, mismatched, or replayed capabilities.

SQL workers use source credentials unavailable to the model. They enforce read-only statements, approved schemas, statement timeouts, row/byte limits, and query tagging. Python workers run without ambient credentials or network by default, with CPU, memory, wall-time, filesystem, and package limits. Chart workers accept structured data and style specifications, not arbitrary code. Every execution writes a hash-addressed result and audit record.

10.3 Repair Loop

Repairs are bounded. The verifier may return a machine-readable reason and permitted repair class, such as rename a superseded column or add a required metric predicate. The runtime may attempt at most the policy limit, preserving every failed plan and execution. Permission, policy, or authority failures are never model-repairable.

11 Verification, Claims, and Publication

11.1 Pre-Execution Verification

For SQL, the verifier parses an AST and checks read-only safety, approved relations and columns, metric-required filters, time semantics, groupings, joins, unsafe functions, row limits, and permission scope. It checks schema version, data freshness, snapshot/watermark state, and whether all referenced objects are in the context manifest. Unsupported constructs produce review or rejection; string matching alone is insufficient.

11.2 Post-Execution Verification

After execution, verify result shape, empty or partial outputs, freshness, numeric consistency, denominator rules, uncertainty requirements, chart-data equality, and output sensitivity. Extract policy-relevant claims from report spans, notebook cells, chart annotations, table cells, dashboard text, or application payloads.

Claim types include numeric, comparison, trend, forecast, causal, and operational recommendation. Each type has a support plan. Numeric claims require query/result, data state, metric, schema, execution hash, and verification. Causal claims require evidence and remain review-gated unless a pre-approved causal design applies. Recommendations require supporting facts, policy, decision authority, and an explicit reviewer or automation rule.

11.3 Claim Dependency Graph

For claim c , amos records edges to computations, data versions, metrics, schemas, documents, feedback, model outputs, verification records, and review decisions:

$$\text{traceable}(c) \not\Rightarrow \text{correct}(c) \not\Rightarrow \text{causally_justified}(c). \quad (8)$$

The graph proves traceability and supports change impact. Verification supports limited consistency. Causal justification remains a separate methodological judgment.

11.4 Publication Linearization Point

Immediately before commit, the runtime revalidates policy epoch and all C1 observations. If any governing state changed, it aborts, repairs, or requests review. The linearization point is the atomic database commit that stores artifact metadata, claims, dependencies, verification, replay descriptor, and audit event. Upload large blobs first under a temporary key; commit their hashes and promote them idempotently after the database transaction.

12 Feedback, Learning, and Invalidation

Human feedback is a governed state transition. A correction records author, role, authority, scope, effective interval, target artifact/claims, supporting decision, applicability conditions, and provenance. It never silently edits the old artifact. The next applicable context compilation can select the correction according to time, scope, and authority.

Source events trigger reverse dependency traversal. A changed metric, schema, document, permission, or data state identifies affected claims. Each claim is rechecked against current policy and may remain valid, become stale, become revoked, require review, or schedule re-analysis. Fan-out is bounded by tenant quotas, risk priority, deduplication keys, and cooldown windows. External notifications occur only after a new commit and policy check.

12.1 Replay Levels

Level	Promise	Required retention
0	Disposable exploration	Audit event only; no regeneration claim.
1	Evidence reconstruction	Object versions, queries, documents, results, and review records.
2	Artifact regeneration	Level 1 plus structured artifact inputs, templates, and deterministic render configuration.
3	Computational replay	Level 2 plus C2 data, code, parameters, tool environment, and result hashes.
4	Audit package	Level 3 plus policy, identity, approvals, signed manifests, and exportable chain of custody.

Table 5: Replay language is precise; model trajectory replay is recorded when possible but is not guaranteed.

13 API and Event Contracts

The first public API is versioned under `/v1`. All mutating requests require an idempotency key. Every response includes request ID, transaction ID where applicable, status, warnings, and a stable error code.

Events use an envelope containing event ID, type, schema version, tenant, source, subject, occurred/observed times, correlation and transaction IDs, payload hash, and trace context. Consumers are idempotent. Required events include `source.version.changed`, `policy.epoch.changed`, `permission.revoked`, `task.state.changed`, `artifact.committed`, `claim.invalidated`, `review.completed`, and `job.dead.lettered`.

Endpoint	Purpose	Contract
POST /tasks	Admit and run or queue analysis	Request/trigger, audience, risk, budgets, required freshness; returns transaction and job IDs.
GET /tasks/{id}	Observe lifecycle	State, current step, warnings, budgets consumed, review obligation; no hidden restricted details.
POST /memory/search	Inspect governed memory	Permission-first typed results and manifest; privileged admin modes are separate.
POST /memory	Create governed memory	Authority and source constrained by caller role; append-only version semantics.
POST /verify/sql	Preflight proposed SQL	Structured pass/warning/reject, rule IDs, repairability, referenced versions.
GET /artifacts/{id}	Retrieve artifact metadata	Publication/validity state, audience, evidence links, replay level.
GET /claims/{id}	Inspect a claim	Exact span, type, support edges, verification, review, current validity.
POST /reviews	Approve, reject, or correct	Reviewer identity, scoped decision, affected claims, feedback object created.
POST /replay/{id}	Reconstruct or replay	Allowed level, retained inputs, status, differences, new audit record.
POST /artifacts/{id}/revalidate	Check current validity	Traverses dependencies and returns per-claim changes.

Table 6: Minimum API surface. Administrative configuration and connector secrets use separate privileged interfaces.

14 Security and Multi-Tenancy

14.1 Trust Boundaries

Retrieved content, model output, user-provided text, and external tool output are untrusted. Identity tokens, policy decisions, capabilities, source credentials, verification rules, commit logic, and audit events remain outside model text. A successful prompt injection may influence a proposed plan; it must not grant a capability, reveal filtered data, write high-authority memory, or publish restricted output.

Threat	Preventive control	Detection/recovery
Cross-tenant leakage	Tenant key on every row, database row-level security, tenant-scoped indexes/keys/caches.	Canary records, access-log analytics, immediate key and token revocation.
Indirect prompt injection	Evidence/policy separation, capability checks, sandboxed tools, output validation.	Malicious-content flags, plan diffs, red-team corpus, quarantine.
Permission-revocation race	Policy epoch and permission revalidation at commit.	Abort commit, invalidate affected claims, audit event.
Memory poisoning	Authority tiers, source namespaces, review gates, immutable versions.	Conflict alerts, provenance inspection, rollback by supersession.
Credential exposure	Secret manager, worker-side credential exchange, no secrets in prompts/logs.	Automated scanning, short TTL, rotation and incident runbook.
Unsafe SQL or code	AST checks, read-only roles, isolated workers, network/resource limits.	Kill switch, query tags, execution audit, dead-letter evidence.
Provenance leakage	Permissioned graph traversal and redacted support views.	Link-access tests, export review, access anomaly alerts.
Denial of service	Admission quotas, budgets, concurrency limits, fan-out controls, circuit breakers.	Queue depth/latency alerts, tenant isolation, graceful degradation.

Table 7: Security is enforced by external controls, not by asking the model to follow policy.

14.2 Tenant Isolation

Start with shared application instances and a shared PostgreSQL cluster using row-level security, tenant-prefixed object keys, tenant-scoped encryption contexts, separate cache namespaces, and per-tenant quotas. Offer dedicated databases, worker pools, and customer-managed keys when required. Every background job restores tenant context explicitly; missing tenant context is a hard failure.

14.3 Privacy and Governance

Minimize retained raw rows. Support field classification, masking, purpose limitation, regional storage, retention, legal hold, export, and erasure. Erasure must update both primary records and derived artifacts while retaining only the minimum audit proof permitted by policy. High-risk model providers receive only the bounded, redacted context allowed by tenant configuration.

15 Deployment and Scalability Architecture

15.1 Initial Production Topology

Deploy the modular monolith as stateless API/runtime replicas behind a load balancer. Use managed PostgreSQL with point-in-time recovery, object storage with versioning, a managed queue, isolated worker pools, secret manager, identity provider, metrics/logs/traces backend, and a private network path to customer sources. Separate development, staging, and production accounts. Infrastructure is declarative and reviewed.

Run interactive and scheduled workloads in separate queues. Partition workers by tool type and risk. Use transaction IDs and idempotency keys across retries. Database transactions own state transitions; queue acknowledgements occur only after durable state is written. Long operations renew leases and checkpoints.

15.2 When to Extract Services

Do not distribute the system prematurely. Extract a component only when one of these conditions is measured: tool workers require a stronger isolation boundary; connector traffic has independent scaling; context indexes exceed primary-database capacity; invalidation fan-out interferes with interactive latency; enterprise deployment requires separate data planes; or team ownership demands an independent release cycle. Preserve the same typed contracts when extracting.

15.3 Scaling Strategy

Scale persistent memory and model context independently. Keep structured filters in the database, move large blobs to object storage, and introduce a dedicated lexical/vector candidate index behind the memory interface when needed. Shard by tenant before sharding within a tenant. Maintain reverse dependency indexes for invalidation. Cache only version-addressed, permission-scoped results; policy-epoch changes evict them.

Capacity planning tracks active tenants, memory objects, version rate, dependency edges, scheduled tasks, tool concurrency, result bytes, model tokens, p50/p95/p99 control-path latency, queue age, and invalidation fan-out. Load tests include one-million-object tenants, policy churn, source delay, and noisy neighbors.

16 Operations, Reliability, and Support

16.1 Service Objectives

Initial paid-pilot objectives are 99.5% API availability, 99% successful scheduled runs excluding declared source outages, p95 admission/context compilation below two seconds at the supported scale, zero confirmed cross-tenant or permission leaks, and 100% of final artifacts with complete claim/evidence manifests. Objectives tighten only after error budgets and workload shape are understood.

16.2 Observability

Every request carries request, transaction, tenant, job, plan, execution, artifact, and trace IDs. Metrics cover task outcomes, verifier results by rule, context role coverage, retrieval leaks, repair counts, review backlog, capability denials, query cost, model cost, queue age, source health, invalidation lag, replay success, and storage growth. Logs are structured, redacted, immutable according to policy, and linked to traces. Customer-visible status distinguishes AMOS failure from source or provider failure.

16.3 Runbooks

Ship runbooks for source authentication failure, permission leak suspicion, policy epoch mismatch, stuck transaction, worker compromise, runaway query, queue backlog, database failover, object-store mismatch, invalidation storm, replay mismatch, model-provider outage, and data erasure. Each runbook specifies trigger, containment, customer impact, evidence preservation, recovery, owner, and post-incident tests.

Backups are restored in staging on a schedule. Target RPO is five minutes and RTO is one hour for the control plane during paid pilot, with a longer declared window for reconstructible artifacts. Disaster-recovery drills verify database, object, queue, key, and connector configuration recovery together.

17 Testing and Quality Gates

Testing mirrors the architecture rather than only happy-path answers.

Layer	Required tests	Release gate
Unit/property	Time intervals, authority ordering, supersession, policy evaluation, state transitions, hash stability.	Deterministic and exhaustive boundary cases; mutation/property tests for core invariants.
Connector contract	Versions, revocation, pagination, retry, cursor recovery, access denial, rate limit, source deletion.	Same suite passes every connector; no adapter-specific bypass.
Context compiler	Role completeness, permission-first filtering, conflicts, ambiguity, budget pressure, compaction lineage.	Zero restricted/superseded selections; mandatory roles never silently dropped.
Verifier	Valid/invalid SQL corpus, metrics, schema drift, freshness, unsafe functions, claim support.	Published precision/recall thresholds and zero acceptance of seeded critical violations.
End to end	Request, schedule, review, feedback, replay, invalidation, abort/retry, artifact export.	Every terminal outcome and recovery path exercised with persisted evidence.
Security	Cross-tenant, prompt injection, poisoning, capability replay, secret leakage, provenance inference, DoS.	Zero critical leaks; red-team findings have regression tests.
Reliability	Worker kill, database failover, duplicate events, lost acknowledgement, source timeout, model outage.	No duplicate publication; idempotent recovery preserves audit trail.
Performance	Memory/edge growth, concurrency, churn, fan-out, noisy neighbor, queue saturation.	SLOs hold at declared paid-pilot limits with recorded capacity margin.
Human review	Evidence usability, defect detection, correction scope, audit time.	Reviewers can locate support and submit bounded corrections without engineer help.

Table 8: A release is not qualified because a demo answer looks correct; each control boundary must pass its own oracle.

17.1 Definition of Done for a Feature

A feature is done only when its typed contract, migration, permissions, audit events, metrics, failure modes, idempotency behavior, retention behavior, tests, operator runbook, and user-facing status are implemented. Model prompt changes alone do not constitute a product feature.

18 Step-by-Step Build Plan

The team advances by exit gates, not calendar optimism.

Phase	Product state	Build	Exit gate
0	Architecture foundation	Domain types, migrations, transaction state machine, audit log, tenant/identity skeleton, CI, local fixtures, one-command environment.	All invariants have executable tests; a failed step is inspectable and idempotently retryable.
1	Governed memory	Memory/version schema, authority/time/supersession, PostgreSQL search, permission policy, seed tooling, admin inspection.	Permission, time, authority, and supersession property tests pass; no destructive update path.

Phase	Product state	Build	Exit gate
2	One connector and context	Warehouse adapter, source version observation, role schema, context compiler, manifest, access pushdown, ambiguity handling.	Frozen customer task set always includes required roles; zero restricted or superseded retrievals.
3	Governed answer	Typed planner, SQL worker, AST verifier, metric/schema/freshness checks, result hashes, cited report, task API.	At least 90% safe-and-correct completion on frozen pilot tasks; every numeric claim resolves to evidence.
4	Persistent analyst	Review UI/API, scoped feedback, claim graph, replay levels 1–2, schedules, source-change invalidation, notifications.	Four weekly shadow cycles complete; corrections affect the next applicable run; stale claims are identified correctly.
5	Paid pilot	SSO/RBAC, multi-tenant isolation, object storage, durable queues, observability, backups, runbooks, quotas, usage metering.	Three design partners; zero critical security findings; recovery drill passes; onboarding under ten business days.
6	Platform API	Stable SDK, connector kit, application templates, model routing, admin controls, retention/erasure, exportable audit packages.	Two independent applications use the same memory/runtime without bespoke core changes.
7	Scale and enterprise	Hybrid indexes, worker/service extraction where measured, dedicated tenancy options, regional deployment, customer-managed keys.	SLOs pass at target tenant/edge/concurrency scale; noisy-neighbor and failover tests pass.
8	Controlled autonomy	Policy-scoped auto-publication, dependency-driven re-analysis, portfolio scheduling, advanced statistics, rollback controls.	Each autonomous task class has a separate safety case, customer approval, monitoring, kill switch, and rollback drill.

Table 9: Implementation sequence from empty repository to controlled autonomy.

18.1 Reference Repository Shape

Preserve architectural boundaries in code before extracting network services. A practical modular-monolith layout is:

apps/api/	admission, public API, event streaming
apps/web/	Memory Studio, workspace, review, operations
amos/domain/	IDs, value objects, states, errors, interfaces
amos/memory/	objects, versions, authority, time, indexes
amos/connectors/	connector SDK and source adapters
amos/context/	role schemas, selection, reconciliation, manifests
amos/runtime/	planner, controller, budgets, checkpoints, repair
amos/workers/	isolated SQL, Python, statistics, chart workers
amos/verification/	policy, SQL, result, claim, publication checks
amos/evidence/	claims, dependencies, artifacts, replay packages
amos/scheduler/	jobs, leases, retries, invalidation, revalidation
amos/policy/	identity, authorization, risk and retention rules
infra/migrations/	forward database migrations and data backfills
deploy/	local, staging, production and recovery definitions
tests/	unit, contract, integration, security, E2E, load

Dependencies point inward to `amos/domain`; domain code imports no framework, database driver, model SDK, or connector. Packages call another package through a typed interface and may not write its tables. Transactional state change and an outbox

event commit together before asynchronous work begins. External adapters implement ports owned by the calling domain. Architecture tests enforce allowed imports, table ownership, tenant-scoped repositories, and the absence of credentials from model-facing packages.

18.2 First Vertical Slice

Use a payment-health or subscription-health review as the first commercial workflow. It is recurring, economically meaningful, bounded, and reviewable. Limit the alpha to one metric family, one warehouse, three query shapes, one chart, one report template, and one reviewer role. Shadow the existing analyst process before replacing any deliverable.

The vertical slice is complete when a user request or schedule produces: admitted transaction; versioned context manifest; verified SQL executions; result and chart hashes; structured claims; review-aware report; claim dependencies; replay package; audit trail; and explicit terminal outcome. A fluent report without these records is not an AMOS product slice.

18.3 Team and Ownership

A five-person initial team is sufficient: founding architect/product lead; memory/connectors engineer; runtime/tools engineer; product/review engineer; and design-partner/data lead. Assign one named owner for security and operations even if the role is fractional. Every architectural component in Table 3 has a code owner and operational owner before paid pilot.

19 Current Prototype to Production

The repository already demonstrates useful seams: Python domain models, SQLite-backed memory, permission filtering, temporal/authority reconciliation, a planner/controller, DuckDB execution, SQL verification, claim provenance, replay, FastAPI routes, scenarios, and an 85-test regression suite. This proves the vertical control loop can run. It is not yet the complete production architecture.

Area	Prototype	Production completion work
State store	SQLite tables and local artifacts.	PostgreSQL migrations, row-level security, HA, PITR, object storage, retention, tenant keys.
Transactions	Controller sequence and audit records.	Explicit atxn state machine, policy/source revalidation, leases, idempotency, atomic evidence commit.
Connectors	Synthetic DuckDB and fixture-shaped adapters.	Versioned connector SDK, real warehouse/IAM/semantic/document adapters, change subscriptions.
Runtime	One payment workflow and templated plan.	Typed multi-step plans, model router, durable checkpoints, capabilities, isolated workers, repair budgets.
Verification	Read-only SQL, schema, metric, freshness, permissions, provenance.	Supported SQL fragment spec, result/statistical checks, rule versioning, claim-type policies, false-reject monitoring.
Provenance	Claim records, support links, replay hashes.	General dependency graph, artifact spans, review decisions, reverse invalidation, signed audit export.
API	FastAPI task/memory/claim/artifact/replay routes.	Versioning, idempotency, pagination, quotas, stable error taxonomy, event schemas, SDK.
Operations	Local execution and evaluation harness.	Queue, observability, SLOs, on-call, backups, DR, metering, customer status, incident runbooks.
Security	Permission filters and read-only tools.	SSO/RBAC, tenant isolation, secret manager, worker sandbox, egress controls, red-team program.

Table 10: The production plan extends the existing prototype without confusing local feasibility with a deployable product.

20 Architecture Decisions and Non-Goals

Key decisions are explicit:

- build a modular monolith first, with internal service contracts and one authoritative transaction database;
- keep raw data in source systems and move bounded results or retained snapshots only under policy;
- enforce permissions before retrieval ranking and again at execution and commit;

- treat models as untrusted, replaceable processors with no ambient credentials;
- use optimistic cross-source validation rather than claim nonexistent global ACID guarantees;
- attach provenance at claim granularity and define replay levels precisely;
- gate causal and high-impact conclusions on human or approved methodological review;
- add autonomy per task class, never as a platform-wide switch.

Non-goals for the first product are universal enterprise knowledge management, unrestricted natural-language access to all data, arbitrary production writes, perfect model reproducibility, automatic causal discovery, and replacing warehouses, semantic layers, catalogs, lineage systems, or BI tools.

21 Harsh Qualification Checklist

Before claiming that AMOS is ready for a paid pilot, answer yes to every item:

1. Can an operator reconstruct why every selected memory object entered the context and why every omitted mandatory role caused a visible outcome?
2. Can permission revocation between retrieval and publication prevent commit without relying on the model?
3. Can every numeric statement in a final artifact resolve to query, result hash, data version, metric, schema, and verification?
4. Can a reviewer correction affect the next applicable task while the original artifact remains immutable?
5. Can a metric or schema change identify and reclassify every dependent claim?
6. Can duplicate requests, events, or retries occur without duplicate publication?
7. Can a worker be killed or compromised without exposing long-lived credentials or corrupting transaction history?
8. Can the team restore database, objects, queue state, keys, and connector configuration within the declared RTO?
9. Can a tenant export its audit package and enforce retention or erasure without another tenant being observable?
10. Can model provider, retrieval backend, or worker implementation change without changing the memory and atxn contracts?
11. Do load, security, connector, failure-injection, and human-review tests pass at the declared supported scale?
12. Does the product save measurable analyst/reviewer time while maintaining zero confirmed critical permission leaks?

A no answer is an architectural gap, not documentation debt. The corresponding phase remains incomplete until the gap has an owner, contract, test, metric, and runbook.

22 Conclusion

AMOS becomes a product when persistent company knowledge, model reasoning, tool execution, verification, human judgment, and artifacts are joined by one enforceable operating contract. Typed analytical memory provides durable state. The bounded context compiler supplies only valid and permitted working memory. atxn makes admission, version observation, capabilities, validation, commit, replay, and invalidation explicit. Claim dependencies make conclusions inspectable and change-aware. External enforcement keeps models useful without making them trusted authorities.

The implementation path is deliberately practical: build one governed read-only workflow in a modular monolith; make every outcome and dependency observable; add durable review, scheduling, tenancy, operations, and connector breadth; then introduce platform APIs and autonomy only when phase-specific gates pass. If the team follows the contracts and qualification checklist in this guide, it can build AMOS end to end without leaving the hardest correctness, security, and operational decisions implicit.